



PRÁCTICA 3

Optimización automática de hiperparámetros
para un modelo de clasificación multi-target
desbalanceado con Scikit-Learn

Curso : Analítica de Datos (SI150V / ST150V)
Docente : Samuel Alonso Oporto Díaz
Grupo : 4
Fecha : 8 de junio del 2026

Integrantes

De la Cruz Orellana, Aleksander	20212099E
León Sánchez, Fransua Mijail	20210019D
López Arana, Edgar Michael	20210254C

Repositorio de código

https://drive.google.com/drive/folders/1p4MXPYVENPGY_9TK0uh8K0s5D-tUMG0D?usp=drive_link

Resumen

Se construyó un procedimiento de aprendizaje automático cuyo objetivo es minimizar el error de clasificación sobre un problema *multi-target* (tres variables objetivo, cada una con tres clases) generado sintéticamente con `scikit-learn`: 60 000 registros, 22 atributos y una distribución de clases deliberadamente desbalanceada (razón mayoritaria/minoritaria de hasta 5.39). El núcleo del trabajo no es un único modelo, sino el *algoritmo de optimización* que calcula automáticamente sus parámetros: se implementaron y compararon cuatro estrategias de búsqueda de hiperparámetros —búsqueda en cuadrícula (*Grid Search*), búsqueda aleatoria (*Random Search*), optimización bayesiana (Optuna/TPE) y un algoritmo genético (DEAP)— evaluadas con validación cruzada de 3 pliegues y la métrica F1-macro promediada sobre los tres *targets*. Los dos optimizadores que además buscan sobre la *elección del algoritmo* (Optuna y el genético) prácticamente empataron en el tope —F1-macro de 0.8335 y 0.8341 en prueba— y ambos seleccionaron la red neuronal (MLP) como mejor algoritmo de aprendizaje entre los cinco evaluados (redes neuronales, árbol de decisión, *random forest*, *naive bayes* y regresión logística), superando ampliamente a la cuadrícula y la búsqueda aleatoria (0.7709 y 0.7655), que quedaron limitadas a un solo modelo. La optimización bayesiana resultó la más *eficiente* (igual calidad en 52.4 s frente a 71.3 s del genético). El análisis de la curva de aprendizaje confirmó que el problema es propenso al sobreaprendizaje cuando se libera la profundidad del modelo (brecha entrenamiento-validación de +0.2736), y de las tres recomendaciones implementadas la ponderación de clases (`class_weight=balanced`) fue la única que mejoró la métrica sobre la línea base (+0.0147), demostrando ser viable.

Palabras clave: optimización de hiperparámetros, clasificación multi-target, desbalanceo, F1-macro, Grid Search, Random Search, optimización bayesiana, algoritmo genético, MLOps, Scikit-Learn.

1 Introducción

El rendimiento de un modelo de aprendizaje supervisado depende tanto de los parámetros que se ajustan durante el entrenamiento (pesos, umbrales, particiones del árbol) como de los *hiperparámetros* que el analista fija antes de entrenar: la profundidad de un árbol, el número de neuronas de una red o la

fuerza de regularización. Estos últimos no se aprenden por descenso del gradiente ni por el criterio de impureza; deben buscarse en un espacio de configuraciones que crece de forma combinatoria. El problema de *calcular automáticamente* esos hiperparámetros es lo que la literatura denomina *Hyperparameter Optimization* (HPO) y constituye el corazón de esta práctica [5, 2].

Antecedentes. La forma más antigua de HPO es la búsqueda en cuadrícula, que evalúa de manera exhaustiva todas las combinaciones de una rejilla discreta. Bergstra y Bengio [2] demostraron empíricamente que, cuando solo unas pocas dimensiones son relevantes, la búsqueda aleatoria encuentra mejores configuraciones que la cuadrícula con el mismo presupuesto, porque no desperdicia evaluaciones sobre dimensiones irrelevantes. Posteriormente, los métodos secuenciales basados en modelos —la optimización bayesiana— propusieron *aprender* de las evaluaciones anteriores para decidir la siguiente, mediante un modelo sustituto y una función de adquisición [3]. El estimador de Parzen estructurado en árbol (TPE) que implementa Optuna [4] pertenece a esta familia. En paralelo, los algoritmos evolutivos abordan el mismo problema como una optimización poblacional inspirada en la selección natural [6]. Todas estas técnicas conviven hoy en frameworks de AutoML y de MLOps.

Procedimiento. En este trabajo (i) se genera un conjunto de datos multi-target desbalanceado con Scikit-Learn [11]; (ii) se define un espacio de búsqueda que abarca no solo los hiperparámetros de cada algoritmo de aprendizaje sino también las decisiones de preprocesamiento (normalización, reducción de dimensionalidad, ponderación de clases) y la elección misma del algoritmo; (iii) se implementan cuatro optimizadores sobre ese espacio; y (iv) se selecciona la mejor configuración y se la somete a un análisis de sobreaprendizaje y a la implementación de recomendaciones.

Resultados esperados. Se espera que la búsqueda aleatoria y la bayesiana superen a la cuadrícula en relación calidad/tiempo, que la métrica F1-macro sea sensible al desbalanceo y que el modelo presente sobreaprendizaje al aumentar su capacidad, lo cual justifica el uso de regularización y ponderación de clases como contramedidas.

2 Planteamiento del problema

2.1 Sistematización de conceptos

El enunciado fija un vocabulario preciso que conviene formalizar. Un **target** es una variable que el modelo debe predecir (una columna de la matriz de salida). El **conjunto de clases** de un target es el conjunto de todos sus valores posibles; una **clase** es uno de esos valores asignado a una instancia (también llamado *estado*), y una **etiqueta** (*label*) es la clase concreta observada en una instancia particular. Con estas definiciones, la Figura 1 resume los cuatro tipos de problema de clasificación: binario (un target, dos clases), multiclase (un target, > 2 clases), multitiqueta (varios labels binarios simultáneos) y multi-target o *multi-output* (varios targets, cada uno con su propio conjunto de clases, predichos conjuntamente).

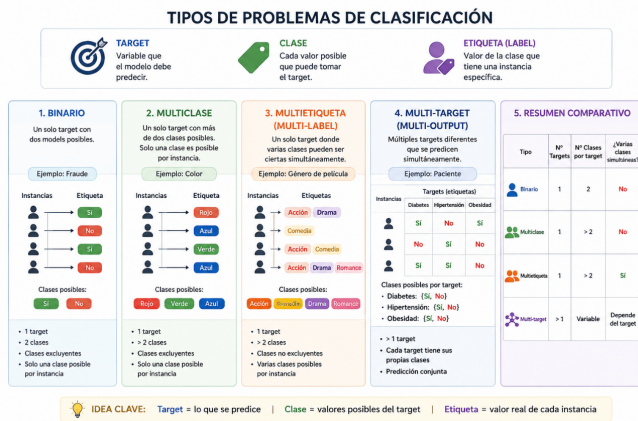


Figura 1: Tipos de problema de clasificación según el número de *targets* y de clases por *target*. Nuestro caso corresponde a la cuarta categoría: multi-target con tres clases por *target*.

2.2 El problema concreto y la decisión sobre los datos

El enunciado pide datos con “al menos 3 labels, al menos 3 clases por cada label, al menos 20 atributos y al menos 50 000 registros”, desbalanceados, generados con `make_multilabel_classification()` o, alternativamente, con `make_classification()`. Existe una incompatibilidad técnica que debe explicitarse: `make_multilabel_classification` produce *labels binarios* (presencia/ausencia), de modo que cada *label* tendría solo dos clases y nunca tres. Por ello, para satisfacer literalmente el requisito de “ ≥ 3 clases por *label*” se utilizó la alternativa explícita del enunciado, `make_classification()`, para construir un problema **multi-target multiclase**: $T = 3$ tar-

gets, cada uno con $K = 3$ clases, sobre una misma matriz de 22 atributos y 60 000 registros. No obstante, para no descartar la función nombrada en primer lugar, también se generó el conjunto multitiqueta y se reprodujeron con datos reales las tablas de contingencia de frecuencias que el enunciado presenta como ejemplo (Sección 6.1).

2.3 Formalización

Sea $\mathbf{x} \in \mathbb{R}^{22}$ el vector de atributos de una instancia y $\mathbf{y} = (y^{(1)}, \dots, y^{(T)})$ su vector de etiquetas, con $y^{(t)} \in \{0, 1, 2\}$. Se busca una función $f_{\lambda} : \mathbb{R}^{22} \rightarrow \{0, 1, 2\}^T$, parametrizada por una configuración de hiperparámetros λ , que minimice el error esperado:

$$\lambda^* = \arg \min_{\lambda \in \Lambda} \mathbb{E}_{(\mathbf{x}, \mathbf{y})} [\mathcal{L}(\mathbf{y}, f_{\lambda}(\mathbf{x}))], \quad (1)$$

donde Λ es el espacio de búsqueda y $\mathcal{L}$ una pérdida de clasificación. Como $\mathbb{E}[\cdot]$ es desconocida, se la aproxima con el promedio de la métrica sobre validación cruzada (Sección 6.3). La Ecuación (1) encierra los *dos* objetivos integrados que señala el enunciado: el objetivo de aprendizaje (minimizar el error de f) y el objetivo de optimización (encontrar el λ que lo minimiza).

3 Metodología

El proyecto se organizó siguiendo el modelo de referencia **MLOps**, adaptado a las fases que realmente se desarrollan en esta práctica (no se incluyen despliegue en producción ni monitoreo en línea porque exceden el alcance del entregable). La Figura 2 muestra el ciclo adoptado: ingesta y generación de datos, validación y análisis exploratorio, preparación, definición del espacio de búsqueda, optimización de hiperparámetros (la pieza central), evaluación y, finalmente, empaquetado del modelo y de los artefactos reproducibles.

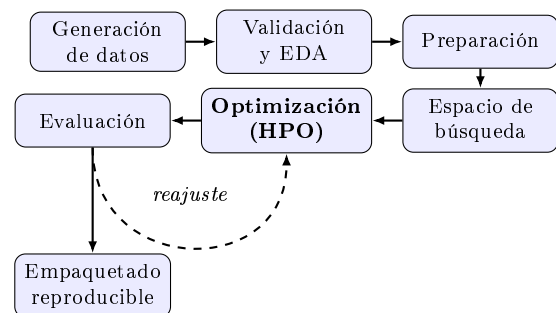


Figura 2: Ciclo MLOps adaptado al proyecto. La optimización de hiperparámetros realimenta a la evaluación hasta converger en la mejor configuración.

Cada fase del ciclo se materializa en un *componente* de software (Sección 6). La trazabilidad se garantiza con una semilla aleatoria fija, el guardado de los datasets y predicciones en CSV, y la exportación de todas las métricas a un archivo `results.json` del que se derivan automáticamente las cifras de este informe.

4 Objetivos

Objetivo general. Construir un procedimiento de aprendizaje que minimice el error de clasificación multi-target mediante un algoritmo de optimización que calcule automáticamente los hiperparámetros del modelo.

Objetivos específicos (componentes). Cada objetivo específico es un componente con insumos y resultados bien definidos (Figura 3):

1. Generar un dataset multi-target desbalanceado que cumpla las restricciones del enunciado.
2. Preparar los datos y definir una métrica adecuada al desbalanceo.
3. Implementar y explicar los cuatro algoritmos de optimización sobre un espacio de búsqueda común.
4. Comparar los optimizadores y seleccionar la mejor configuración (modelo + preprocesamiento).
5. Diagnosticar el sobreaprendizaje y proponer e implementar recomendaciones.

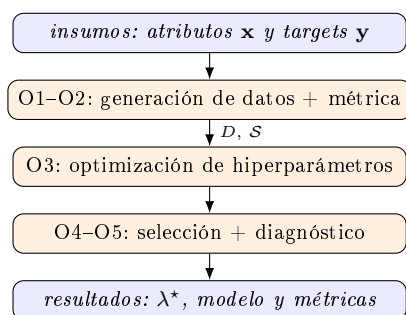


Figura 3: Encadenamiento de objetivos como componentes. El flujo entra por los insumos (arriba), atraviesa cada objetivo —una caja por componente— y entrega los resultados (abajo) que alimentan al siguiente.

5 Arquitectura del sistema

Se describe la arquitectura en tres niveles de detalle crecientes.

6 Construcción de los componentes

Cada componente se presenta con un fragmento del código fuente real, el resultado de su corrida y la explicación de lo realizado. El código completo se entrega comprimido junto al informe.

6.1 C1. Generación del dataset

El *target* 1 se generó directamente con `make_classification` (tres clases con pesos 0,62/0,27/0,11). Los *targets* 2 y 3 se construyeron como funciones —una de ellas no lineal— de las doce primeras variables informativas, discretizadas por cuantiles para inducir desbalanceo. Así los tres *targets* dependen genuinamente de los mismos atributos y son aprendibles.

Listing 1: C1 – Generación multi-target multiclase desbalanceada

```

1 X, y1 = make_classification(
2     n_samples=60000, n_features=22, n_informative=12,
3     n_redundant=4, n_classes=3, n_clusters_per_class
4     =2,
5     weights=[0.62, 0.27, 0.11], class_sep=0.9,
6     flip_y=0.02, shuffle=False, random_state=SEED)
7
8 inf = X[:, :12]
9
10 def build_target(seed, cuts, nonlin=False):
11     w = np.random.RandomState(seed).randn(inf.shape
12         [1])
13     score = inf @ w
14     if nonlin:
15         score += 0.6*(inf[:,0]*inf[:,3]) - 0.5*(inf
16            [:,5]**2)
17     score += rng.normal(0, score.std()*0.35, len(
18         score))
19     return np.digitize(score, np.quantile(score, cuts
20     ))
21
22 y2 = build_target(101, [0.55, 0.85], nonlin=True)
23 y3 = build_target(202, [0.50, 0.78])
24 Y = np.column_stack([y1, y2, y3])
  
```

La Tabla 1 y la Figura 7 confirman el desbalanceo buscado: la clase mayoritaria del *target* 1 es 5.39 veces más frecuente que la minoritaria. Este desbalanceo es lo que obliga a elegir una métrica robusta (Sección 6.3).

El diseño se comprobó con tres pruebas de variedad. Primero, las **27** combinaciones posibles de $(y^{(1)}, y^{(2)}, y^{(3)})$ aparecen en los datos (la menos frecuente con 204 instancias), de modo que no hay estados degenerados. Segundo, los tres *targets* son casi **independientes** entre sí (información mutua normalizada inferior a 0,003): cada uno es un problema de predicción distinto y no una copia de otro. Tercero, cada *target* se apoya en un **subconjunto diferente** de atributos —la información mutua máxima atributo-objetivo va de 0,06 a 0,16 según el

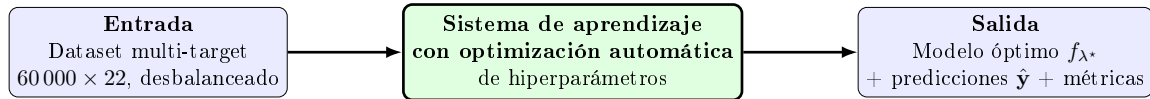


Figura 4: Arquitectura de Nivel 0 (caja negra). El sistema recibe el dataset desbalanceado y entrega el modelo óptimo, sus predicciones y sus métricas de desempeño.

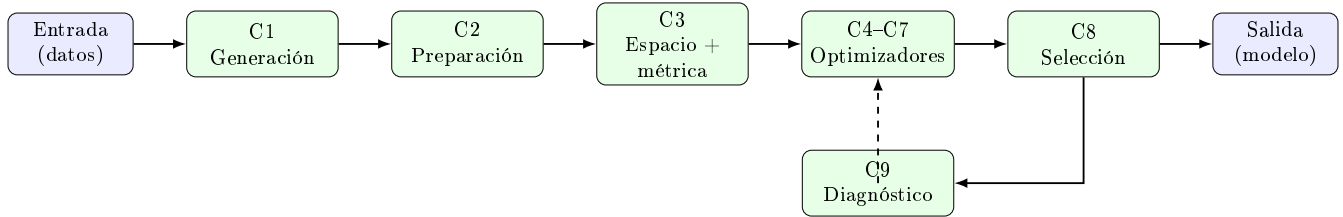


Figura 5: Arquitectura de Nivel 1. La caja negra se abre en la cadena de componentes del *pipeline*; cada componente deja un artefacto persistente (CSV/PNG/JSON) que alimenta al siguiente.

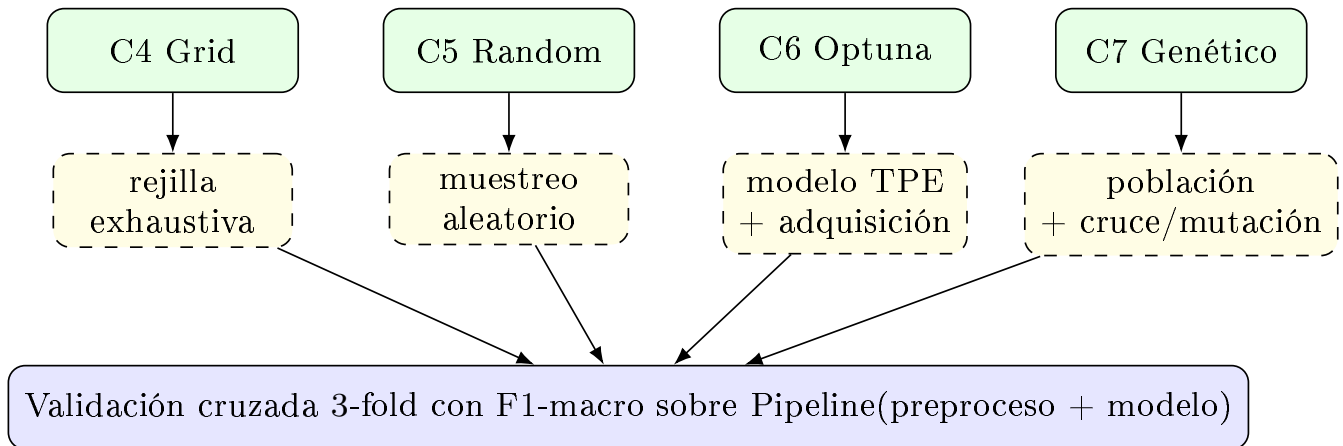


Figura 6: Arquitectura de Nivel 2. Detalle de los sub-procedimientos de cada optimizador; todos comparten el mismo evaluador: validación cruzada con F1-macro sobre un *pipeline* que encadena preprocesamiento y modelo.

target—, lo que evita un dataset trivial y mantiene el problema genuinamente multivariado.

Tabla 1: Distribución de clases por *target* (% de registros) y razón de desbalanceo (RD).

Target	clase 0	clase 1	clase 2	RD
target1	61.4 %	27.1 %	11.4 %	5.39
target2	55.0 %	30.0 %	15.0 %	3.67
target3	50.0 %	28.0 %	22.0 %	2.27

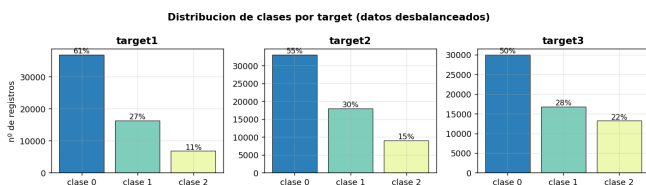


Figura 7: Distribución de frecuencias de clase por *target*. Las barras evidencian la asimetría (clase 0 dominante) que caracteriza a un problema desbalanceado.

dujo el ejemplo de tablas de contingencia del enunciado. La Figura 8 muestra la co-ocurrencia real de *label1* y *label2*: la celda (1,1) concentra la mayor masa de probabilidad, exactamente el patrón de desbalanceo conjunto que ilustra el enunciado.

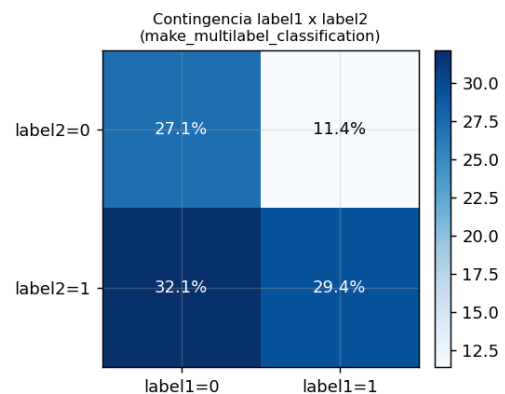


Figura 8: Tabla de contingencia *label1*×*label2* (% del total) obtenida con `make_multilabel_classification`, réplica con datos reales del ejemplo del enunciado.

Adicionalmente, `make_multilabel_classification` se repro-

6.2 C2. Preparación de datos

Se dividió el conjunto en 48 000 registros de entrenamiento y 12 000 de prueba (80/20). Para acelerar la *fase de búsqueda* —que repite cientos de ajustes— se trabajó sobre una submuestra de 12 000 registros; el modelo ganador se reajusta luego sobre el entrenamiento completo. El propio tamaño de la muestra de entrenamiento es, de hecho, uno de los parámetros del proceso de aprendizaje que el enunciado pide considerar, y su efecto se cuantifica en la curva de aprendizaje (Sección 6.9). La normalización (`StandardScaler`/`MinMaxScaler`/ninguna) no se fija a priori sino que se incorpora como una decisión más del espacio de búsqueda.

6.3 C3. Espacio de búsqueda y métrica

Métrica. Con clases desbalanceadas, la exactitud (*accuracy*) es engañosa: un clasificador que prediga siempre la clase mayoritaria del *target* 1 lograría $\approx 61\%$ de aciertos sin aprender nada. Por eso se adopta el **F1-macro**, media no ponderada del F1 de cada clase, que penaliza el mal desempeño en clases minoritarias. Para el caso multi-target se promedia además sobre los T targets:

$$F1_{\text{macro}} = \frac{1}{T} \sum_{t=1}^T \frac{1}{K} \sum_{k=1}^K \frac{2 P_k^{(t)} R_k^{(t)}}{P_k^{(t)} + R_k^{(t)}}, \quad (2)$$

donde P y R son precisión y *recall*. Se reportan también la *Hamming loss* (fracción de etiquetas mal predichas) y la *subset accuracy* (aciertan los tres *targets* a la vez). Toda la evaluación se realiza con validación cruzada de Scikit-Learn.¹

Listing 2: C3 – Métrica F1-macro multi-target como scorer de Scikit-Learn

```
1 def f1_macro_multioutput(y_true, y_pred):
2     return np.mean([
3         f1_score(y_true[:, j], y_pred[:, j], average="macro")
4         for j in range(y_true.shape[1])]
5
6 SCORER = make_scorer(f1_macro_multioutput)
```

Espacio de búsqueda. El espacio Λ es un *pipeline* configurable que recoge los parámetros del proceso de aprendizaje que enumera el enunciado: el *tamaño de la muestra de entrenamiento* (Sección 6.9), la *técnica de normalización* (escalador estándar, min-max o ninguno), el *número de atributos* que entran al modelo (vía PCA), el *balanceo*

de clases por *target* (`class_weight`), la *elección del algoritmo* de aprendizaje entre los cinco solicitados y los *hiperparámetros* propios de cada uno (Tabla 2). Los algoritmos sin soporte multi-target nativo (*naive bayes*, regresión logística) se envuelven en `MultiOutputClassifier`, que ajusta un estimador independiente por *target*.

Tabla 2: Hiperparámetros optimizados por algoritmo de aprendizaje.

Algoritmo	Hiperparámetros
Random Forest	<code>n_estimators</code> , <code>max_depth</code> , <code>min_samples_split</code> , <code>max_features</code> , <code>class_weight</code>
Árbol decisión	<code>max_depth</code> , <code>min_samples_split</code> , <code>min_samples_leaf</code> , <code>criterion</code> , <code>class_weight</code>
Red neuronal (MLP)	<code>hidden_layer_sizes</code> , <code>activation</code> , <code>alpha</code> , <code>learning_rate_init</code>
Naive Bayes Reg. logística	<code>var_smoothing</code> <code>C</code> , <code>class_weight</code>
Preprocesamiento	escalador $\in \{\text{standard, minmax, none}\}$, PCA $\in \{1.0, 0.95, 0.85\}$

6.4 C4. Búsqueda en cuadrícula (Grid Search)

Cómo opera. La búsqueda en cuadrícula define un conjunto finito de valores por hiperparámetro y evalúa el *producto cartesiano* de todos ellos; es exhaustiva y garantiza encontrar el óptimo *dentro de la rejilla*, pero su costo crece exponencialmente con el número de dimensiones (la “maldición de la dimensionalidad”) [2]. Se aplicó sobre *random forest* con la clase `GridSearchCV` de Scikit-Learn.²

Listing 3: C4 – Grid Search exhaustivo con validación cruzada

```
grid = {"model__n_estimators": [100, 200],
        "model__max_depth": [20, None],
        "model__min_samples_split": [2, 10],
        "model__max_features": ["sqrt", "log2"]}
gs = GridSearchCV(pipe_rf, grid, scoring=SCORER,
                  cv=3, n_jobs=-1).fit(Xs, Ys)
```

Resultado. 16 combinaciones, mejor F1-macro en validación de 0.7296 en 26.3 s.

6.5 C5. Búsqueda aleatoria (Random Search)

Cómo opera. En lugar de recorrer una rejilla, la búsqueda aleatoria *muestrea* configuraciones de distribuciones definidas sobre cada hiperparámetro y evalúa un presupuesto fijo de n muestras. Su ventaja, demostrada por Bergstra y Bengio [2], es que cubre mejor las dimensiones relevantes cuando no

¹https://scikit-learn.org/1.4/tutorial/statistical_inference/model_selection.html

²https://scikit-learn.org/stable/modules/grid_search.html

todas lo son, a igual número de evaluaciones que la cuadrícula.

Listing 4: C5 – Random Search sobre distribuciones de hiperparámetros

```

1  rand_space = {"model__n_estimators": randint(80, 320)
2              ,
3              "model__max_depth": [10, 20, 30, None],
4              "model__min_samples_split": randint(2,
5              25),
6              "model__min_samples_leaf": randint(1,
7              15),
8              "model__max_features": ["sqrt", "log2"]
9              }
10 rs = RandomizedSearchCV(pipe_rf, rand_space, n_iter
11 =25,
12 scoring=SCORER, cv=3, n_jobs=-1,
13 random_state=SEED).fit(Xs, Ys)

```

Resultado. 25 muestras, mejor F1-macro de 0.7115 en 40.9 s.

6.6 C6. Optimización bayesiana (Optuna/TPE)

Cómo opera. La optimización bayesiana construye un modelo probabilístico sustituto de la relación $\lambda \rightarrow$ métrica y usa una función de adquisición para elegir el siguiente λ que mejor equilibra exploración y explotación [3]. Optuna implementa el TPE, que modela las densidades de configuraciones “buenas” y “malas” y muestrea donde su cociente es máximo [4]. A diferencia de C4 y C5, aquí el espacio incluye la *elección del algoritmo* y del preprocesamiento (búsqueda condicional), convirtiendo el problema en uno de AutoML.

Listing 5: C6 – Objetivo Optuna sobre el espacio AutoML condicional

```

1  def objective(trial):
2      algo = trial.suggest_categorical("algorithm",
3      ALGORITHMS)
4      scaler = trial.suggest_categorical("scaler",
5      SCALERS)
6      pca = trial.suggest_categorical("pca", [1.0,
7      0.95, 0.85])
8      hp = sample_hp_from_trial(trial, algo)
9      pipe = build_pipeline(algo, scaler, pca, hp)
10     return cross_val_score(pipe, Xs, Ys,
11     scoring=SCORER, cv=3, n_jobs=-1).mean()
12
13 study = optuna.create_study(direction="maximize",
14     sampler=optuna.samplers.TPESampler(seed=SEED))
15 study.optimize(objective, n_trials=40)

```

Resultado. 40 *trials*, mejor F1-macro de 0.8109 en 52.4 s; el algoritmo seleccionado fue mlp.

6.7 C7. Algoritmo genético (DEAP)

Cómo opera. Un algoritmo genético mantiene una *población* de configuraciones codificadas como genomas; en cada *generación* evalúa su aptitud (la

métrica), selecciona a los mejores por torneo y genera descendencia mediante *cruce* y *mutación* [6]. Se exploró el mismo espacio AutoML que en C6 codificando cada configuración en un vector continuo decodificado a hiperparámetros.

Listing 6: C7 – Bucle evolutivo con DEAP

```

1  toolbox.register("mate", tools.cxBlend, alpha=0.4)
2  toolbox.register("mutate", tools.mutGaussian,
3      mu=0, sigma=0.18, indpb=0.3)
4  toolbox.register("select", tools.selTournament,
5      tournsize=3)
6  for gen in range(1, NGEN + 1):
7      offspring = algorithms.varAnd(pop, toolbox,
8      cxpb=0.6, mutpb=0.3)
9      # evaluar aptitud (F1-macro CV) y seleccionar
10     pop = toolbox.select(offspring, k=POP)

```

Resultado. Población de 16 individuos durante 6 generaciones (59 evaluaciones únicas con caché), mejor F1-macro de 0.8097 en 71.3 s; algoritmo elegido: mlp.

6.8 C8. Comparación y selección

La Tabla 3 resume el desempeño de los cuatro optimizadores tras reajustar la mejor configuración de cada uno sobre el entrenamiento completo y evaluarla en prueba. La Figura 9 contrasta calidad y costo, la Figura 10 muestra la convergencia y la Figura 11 el mejor F1 alcanzable con cada algoritmo de aprendizaje.

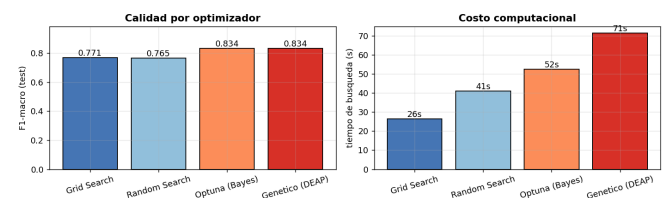


Figura 9: Calidad (F1-macro en prueba) frente a costo (tiempo de búsqueda) de cada optimizador.

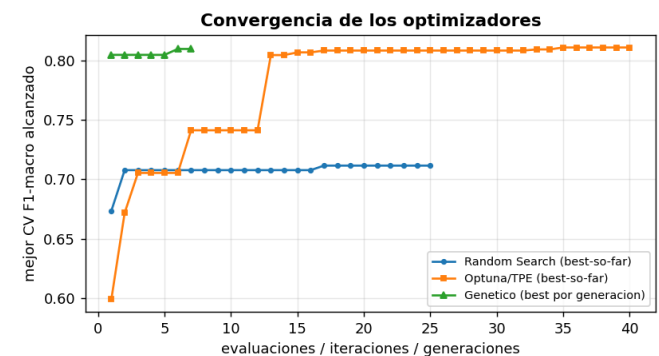


Figura 10: Curvas de convergencia: mejor F1-macro acumulado conforme avanzan las evaluaciones (Random, Optuna) o las generaciones (genético).

Tabla 3: Comparación de los cuatro optimizadores. F1-macro en validación cruzada y en prueba, brecha de sobreaprendizaje (train–test), *Hamming loss*, *subset accuracy* y tiempo de búsqueda.

Optimizador	F1 CV	F1 test	Brecha	Hamming	Subset acc.	Tiempo (s)
Grid Search	0.7296	0.7709	+0.2289	0.1845	0.5427	26.3
Random Search	0.7115	0.7655	+0.1822	0.1872	0.5402	40.9
Optuna (TPE)	0.8109	0.8335	+0.0063	0.1424	0.6273	52.4
Genético (DEAP)	0.8097	0.8341	+0.0055	0.1426	0.6267	71.3

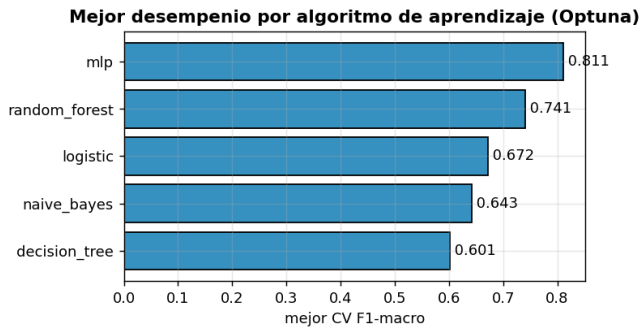


Figura 11: Mejor F1-macro en validación alcanzado por cada algoritmo de aprendizaje durante la búsqueda bayesiana. mlp domina; decision_tree queda rezagado.

La Figura 12 muestra las matrices de confusión del mejor modelo por *target*: la diagonal concentra la masa, pero la confusión residual se da hacia la clase mayoritaria, efecto típico del desbalanceo.

6.9 C9. Diagnóstico de sobreaprendizaje

¿Es el problema propenso al sobreaprendizaje? La evidencia es cuantitativa. Cuando se libera la profundidad del *random forest*, el F1-macro en entrenamiento alcanza 1.0 mientras que en validación se estanca en 0.7264: una brecha de +0.2736 (Figura 13). Esa diferencia es la firma del sobreaprendizaje: el modelo memoriza el ruido (`flip_y=0.02`) y las particiones espurias de las variables redundantes. La curva también muestra que la brecha se reduce al aumentar el tamaño de la muestra, lo que indica que *más datos* y *menor capacidad* son las palancas correctas.

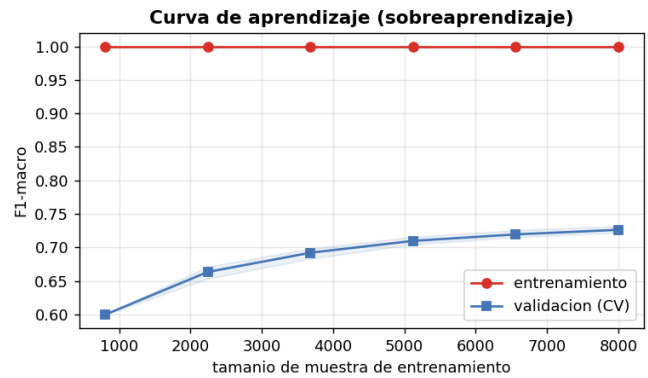


Figura 13: Curva de aprendizaje del mejor modelo. La separación persistente entre entrenamiento y validación revela sobreaprendizaje; la brecha decrece con más datos.

El análisis de PCA (Figura 14) refuerza el diagnóstico: bastan 16 de los 22 componentes para explicar el 95 % de la varianza, señal de redundancia entre atributos que un modelo de alta capacidad puede sobreajustar.

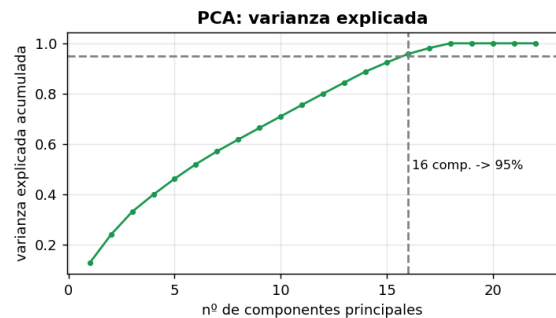


Figura 14: Varianza explicada acumulada por PCA. Con 16 componentes se alcanza el 95 %.

7 Conclusiones

Las conclusiones se expresan como métricas, no como opiniones.

- **Sobre O3–O4 (optimización).** Los optimizadores que buscan también sobre la elección del algoritmo dominaron: el genético alcanzó el mayor F1-macro en prueba (0.8341) y Optuna lo igualó prácticamente (0.8335, diferencia de apenas 0.0006) en menos tiempo (52.4 s frente a

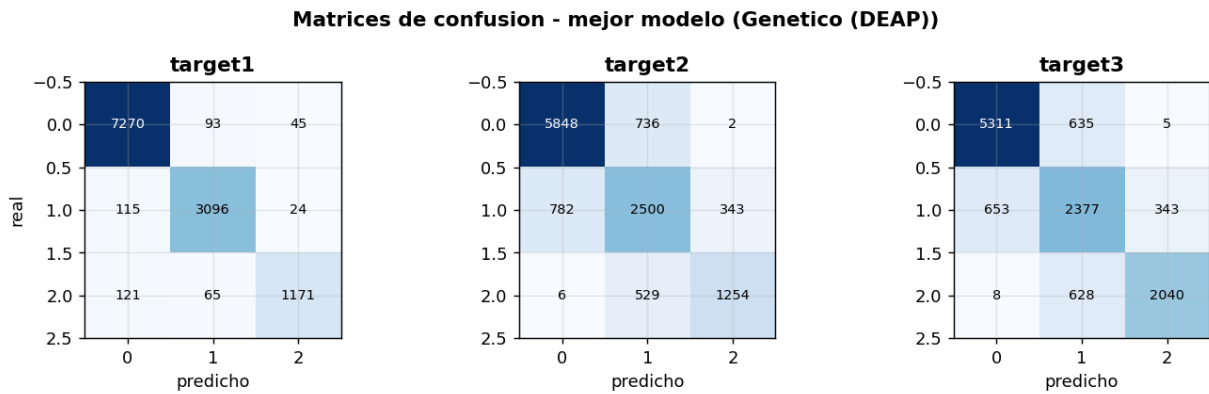


Figura 12: Matrices de confusión del mejor modelo (Genetico (DEAP)) sobre el conjunto de prueba, un panel por *target*.

71.3 s). Ambos superaron por más de 6 puntos a la cuadrícula (0.7709, 26.3 s) y a la búsqueda aleatoria (0.7655, 40.9 s), que quedaron ancladas a *random forest*. La optimización bayesiana es, por tanto, la más eficiente (mejor calidad por unidad de tiempo).

- **Sobre el algoritmo de aprendizaje.** Entre los cinco algoritmos, mlp obtuvo el mayor F1-macro en validación (0.8109) y decision_tree el menor (0.6013); la elección del algoritmo (diferencia de 0.8109 vs 0.6013) pesa más que el ajuste fino de un único modelo, lo que explica por qué Grid y Random —fijados a *random forest*— no alcanzaron el tope.
- **Sobre el objetivo general (minimizar el error).** El mejor procedimiento alcanzó una *Hamming loss* de 0.1424 y una *subset accuracy* de 0.6273 sobre 12 000 registros de prueba nunca vistos.
- **Sobre el sobreaprendizaje.** El problema es propenso a él: la brecha entrenamiento–validación llega a +0.2736 con capacidad alta, y se atenúa con más datos y regularización.
- **Sobre el costo.** El tiempo total de las cuatro búsquedas fue de 190.9 s sobre 12 000 registros, lo que vuelve viable repetir la optimización en cada ciclo de reentrenamiento.

8 Recomendaciones

Se proponen y se *implementan* tres recomendaciones; su viabilidad se decide por métrica (Tabla 4, Figura 15).

1. **Ponderar las clases** (`class_weight=balanced`) para atacar el desbalanceo. Resultado: F1-macro

0.7856 (+0.0147 sobre la línea base) sin costo de preprocesamiento adicional. **viable**.

2. **Reducir dimensionalidad con PCA al 95 %** (16 componentes) para acelerar la inferencia. Resultado: F1-macro 0.7216 (-0.0493); intercambia algo de calidad por velocidad, por lo que resulta **no viable** bajo el criterio adoptado.
3. **Aumentar la capacidad del modelo** (más árboles y mayor profundidad). Resultado: F1-macro 0.7717 (+0.0008); el beneficio marginal no compensa el mayor costo y riesgo de sobreaprendizaje, por lo que es **no viable**.

Tabla 4: Recomendaciones implementadas y su impacto frente a la línea base.

Propuesta	F1 test	Brecha	t (s)	Viable
Baseline (RF)	0.7709	+0.2289	3.6	—
R1: <code>class_weight=balanced</code>	0.7856	+0.2044	3.1	Sí
R2: PCA 95 %	0.7216	+0.2777	3.8	No
R3: mayor capacidad	0.7717	+0.2214	6.7	No

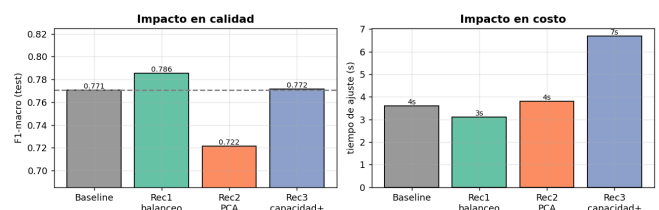


Figura 15: Impacto de cada recomendación en calidad (izq.) y costo de ajuste (der.) respecto de la línea base (línea discontinua).

En síntesis, solo la primera recomendación es plenamente viable: mejora la métrica atacando la causa

raíz (el desbalanceo) sin penalización de costo.

Referencias

- [1] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. DOI: 10.1023/A:1010933404324.
- [2] J. Bergstra and Y. Bengio, “Random Search for Hyper-Parameter Optimization,” *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012. DOI: 10.5555/2188385.2188395.
- [3] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. de Freitas, “Taking the Human Out of the Loop: A Review of Bayesian Optimization,” *Proceedings of the IEEE*, vol. 104, no. 1, pp. 148–175, 2016. DOI: 10.1109/JPROC.2015.2494218.
- [4] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A Next-generation Hyperparameter Optimization Framework,” in *Proc. 25th ACM SIGKDD Int. Conf. on Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631. DOI: 10.1145/3292500.3330701.
- [5] M. Feurer and F. Hutter, “Hyperparameter Optimization,” in *Automated Machine Learning*, Springer, 2019, pp. 3–33. DOI: 10.1007/978-3-030-05318-5_1.
- [6] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989. ISBN: 978-0201157673.
- [7] G. Tsoumakas and I. Katakis, “Multi-Label Classification: An Overview,” *International Journal of Data Warehousing and Mining*, vol. 3, no. 3, pp. 1–13, 2007. DOI: 10.4018/jdwm.2007070101.
- [8] J. Read, B. Pfahringer, G. Holmes, and E. Frank, “Classifier Chains for Multi-label Classification,” *Machine Learning*, vol. 85, no. 3, pp. 333–359, 2011. DOI: 10.1007/s10994-011-5256-5.
- [9] K. Pearson, “On Lines and Planes of Closest Fit to Systems of Points in Space,” *Philosophical Magazine*, vol. 2, no. 11, pp. 559–572, 1901. DOI: 10.1080/14786440109462720.
- [10] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. Springer, 2009. ISBN: 978-0387848570.
- [11] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 2nd ed. O’Reilly, 2019. ISBN: 978-1492032649.